



Integration Team

Interview Project: eCommerce Order Management System

In this project, you will develop a distributed microservices system for an eCommerce platform's order management domain in Java using the Spring Boot framework. The project is designed to be completed in 3-5 days and evaluates your ability to design, implement, and deliver production-ready microservices with enterprise-grade quality standards.

Evaluation Criteria:

- **Well-rounded perspective** Evidence of considering scalability, maintainability, and production concerns
- **Technical expertise:** Deep understanding of Java, Spring, and microservices patterns
- **Quality focus:** High code quality, comprehensive testing, and proper documentation
- **Problem-solving skills:** Creative solutions and well-reasoned technical decisions
- **Production readiness:** Code that could realistically run in a production environment

Core Requirements

1. Order Management Service

Purpose: Core service managing orders and their lifecycle with proper business logic separation.

Functionality:

- CRUD operations with proper business validation
- Order status lifecycle management (unprocessed → processing → processed → shipped)
- Automatic status progression with configurable timing to simulate order phases
- Business rule validation (inventory checks)

- Audit trail for order modifications

Notes:

Technical Implementation:

- PostgreSQL with proper schema design and indexing
- RESTful API with appropriate HTTP status codes
- Input validation and error handling with meaningful responses
- Transaction management for data consistency

API Endpoints:

POST /api/v1/orders - Create order with order lines

GET /api/v1/orders/{id} - Retrieve order with details

PUT /api/v1/orders/{id} - Update order (with business rules)

DELETE /api/v1/orders/{id} - Soft delete order

GET /api/v1/orders - List orders with filtering/pagination

PATCH /api/v1/orders/{id}/status - Update order status

GET /api/v1/orders/{id}/history - Order audit trail

Domain Model (an example, you can design your own model):

- **Order:** orderId, customerId, status, orderDate, totalAmount
- **OrderLine:** productId, quantity, unitPrice, lineTotal
- **Order-OrderLine:** One-to-many relationship with proper cascade handling

Inventory Check note:

Create a simple table in PostgreSQL with the columns productId, availableStock and populate it with sample data to use in business rule validations of order processing.

2. Event Processing & Integration Layer

Purpose: Handle inter-service communication and event-driven architecture patterns.

Functionality:

- Publish order events (OrderCreated, OrderUpdated, OrderProcessed)

- Event sourcing for order status changes
- Implement one integration pattern (message queues or event streaming)

3. Notification Service

Purpose: Demonstrate async processing, external integrations, and resilience patterns.

Functionality:

- Subscribe to order events and send notifications
- Multiple notification channels (email simulation, SMS simulation)
- Failed notification retry with exponential backoff

4. Analytics & Reporting Service

Purpose: Show data aggregation, CQRS patterns, and non-relational database usage.

Functionality:

- Process order events to generate business metrics
- Store aggregated data in MongoDB
- Provide reporting endpoints for business intelligence

Technical Requirements

Architecture & Design

- Apply SOLID principles and appropriate design patterns
- Implement proper layered architecture (Controller/Service/Repository/Domain)
- Use dependency injection effectively
- Design for scalability and maintainability

Code Quality Standards

- Follow Java best practices and modern Java features (Java 11+)
- Implement comprehensive error handling strategy
- Use appropriate data structures and algorithms
- Demonstrate understanding of concurrency and thread safety

Spring Framework Excellence

- Leverage Spring Boot ecosystem effectively
- Implement proper configuration management with profiles
- Implement custom validators and exception handlers

- Use Spring Data JPA with custom queries and specifications

Testing Strategy

- Unit tests demonstration using JUnit
- Integration tests demonstration for database and API layers using TestContainers
- Test data builders and proper test organization

Production Readiness

- Structured logging with correlation IDs across service boundaries
- Health checks and monitoring endpoints
- Configuration externalization with environment-specific settings
- Database connection pooling and optimization

Documentation & Communication

- Comprehensive README with setup instructions and architecture overview
- API documentation (OpenAPI/Swagger)
- Database schema documentation

Delivery Guidelines

What to Submit

- Complete source code with proper Git history
- Docker Compose setup with all required infrastructure (You can start with the sample provided and extend it to meet your needs)
- Comprehensive documentation including setup instructions

Optional Enhancements (Showcase Opportunities)

- API versioning strategy
- Dead letter queue handling
- Rate limiting and throttling
- Caching strategies (Redis integration)
- Advanced monitoring and alerting setup
- Architecture diagrams
- Test reports and coverage information
- Database migration scripts using Liquibase or Flyway

- Spring Security for API protection

Remember: This project is an opportunity to showcase your expertise and engineering judgment. We're more interested in seeing how you think and solve problems than in checking every box. Focus on demonstrating the depth of your experience and the quality of your engineering practices.